Лабораторная работа №5

Моделирование параллельных систем с помощью сетей Петри на примере задачи об обедающих философах

Еремина Оксана

Содержание

1. Цель работы

1. Построить сеть Петри для задачи об обедающих философах.
2. Обнаружить deadlock в классической модели.
3. Модифицировать сеть введением арбитра для предотвращения deadlock.
4. Провести стохастическое моделирование с помощью алгоритма Гиллеспи.
5. Сравнить поведение двух моделей и визуализировать результаты.

2. Теоретическое введение

2.1 Сети Петри

Сеть Петри — математический аппарат для моделирования дискретных систем с параллелизмом и синхронизацией. Формально определяется как четвёрка:

$$N = (P, T, F, M_0),$$

где:

- P — множество позиций (состояния, ресурсы);
- T — множество переходов (события, действия);
- F — множество дуг между позициями и переходами;
- M_0 — начальная маркировка (распределение фишек).

Графически позиции обозначаются кругами, переходы — прямоугольниками. Фишки внутри позиций показывают текущее состояние системы. Переход срабатывает, если

во всех входных позициях достаточно фишек. При срабатывании фишки изымаются из входных позиций и добавляются в выходные.

Ключевые свойства сетей Петри:

- **Ограниченность** — фишек в позиции не больше заданного предела.
- **Активность** — отсутствие тупиков, каждый переход может сработать.
- **Достижимость** — возможность попасть из начального состояния в целевое.

2.2 Задача об обедающих философах

Сформулирована Эдсгером Дейкстрой в 1965 году для иллюстрации проблем синхронизации в параллельных системах.

Постановка:

- N философов сидят за круглым столом.
- Между соседями лежит одна вилка (всего N вилок).
- Философ может думать, быть голодным или есть.
- Чтобы поесть, нужны две вилки — слева и справа.

Проблема deadlock:

Если каждый философ возьмёт левую вилку и будет ждать правую, все заблокируются — система замирает.

Решение с арбитром:

Вводится дополнительный ресурс (арбитр), ограничивающий число одновременно едящих философов до N-1, что предотвращает deadlock.

2.3 Алгоритм Гиллеспи

Стохастический метод моделирования, где время до следующего события и выбор перехода определяются случайным образом на основе интенсивностей переходов.

3. Выполнение лабораторной работы

Создала необходимые файлы, куда скопировала весь код, предоставленный в лабораторной работе.

Запустила их всех, сначала пробежав `install_packages.jl`, установив необходимые библиотеки

Далее создала литературный код для всех основных файлов. После скомпилировала чистый код, jupiter notebook и quarto с помощью файла с кодом `tangle.jl`

4. Моделирование задачи об обедающих философах с помощью сетей Петри

4.1 Введение

Данный скрипт выполняет стохастическое моделирование двух вариантов сети Петри для классической задачи об обедающих философах: 1. Классическая модель (без арбитра) — подвержена deadlock 2. Модифицированная модель с арбитром — предотвращает deadlock

4.2 Параметры моделирования

- `N = 5` — количество философов
- `tmax = 50.0` — время моделирования

4.3 Моделирование классической сети (без арбитра)

В классической постановке каждый философ пытается взять сначала левую вилку, затем правую. Это может привести к взаимной блокировке (deadlock).

```
println("=== Классическая сеть (без арбитра) ===")
net_classic, u0_classic, _ = build_classical_network(N)
```

4.4.1 Стохастическая симуляция

Используется алгоритм Гиллеспи для стохастического моделирования.

```
df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
```

4.4.2 Визуализация эволюции маркировки

Строятся графики для всех позиций сети: Think, Hungry, Eat, Fork (рис. 8).

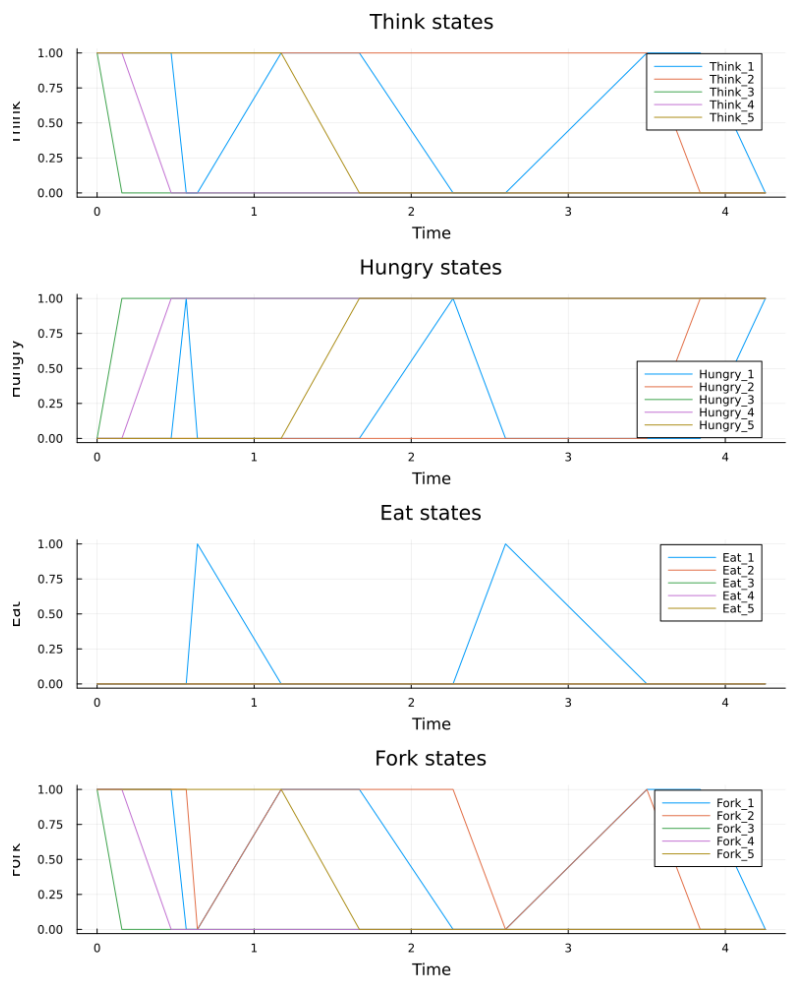

Рисунок 8: График

4.4.3 Стохастическая симуляция

```
df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
```

4.4.4 Визуализация эволюции маркировки

```
plot_arb = plot_marking_evolution(df_arb, N)
savefig(plotsdir("arbiter_simulation.png"))
```

Посмотрим на график (рис. 9)

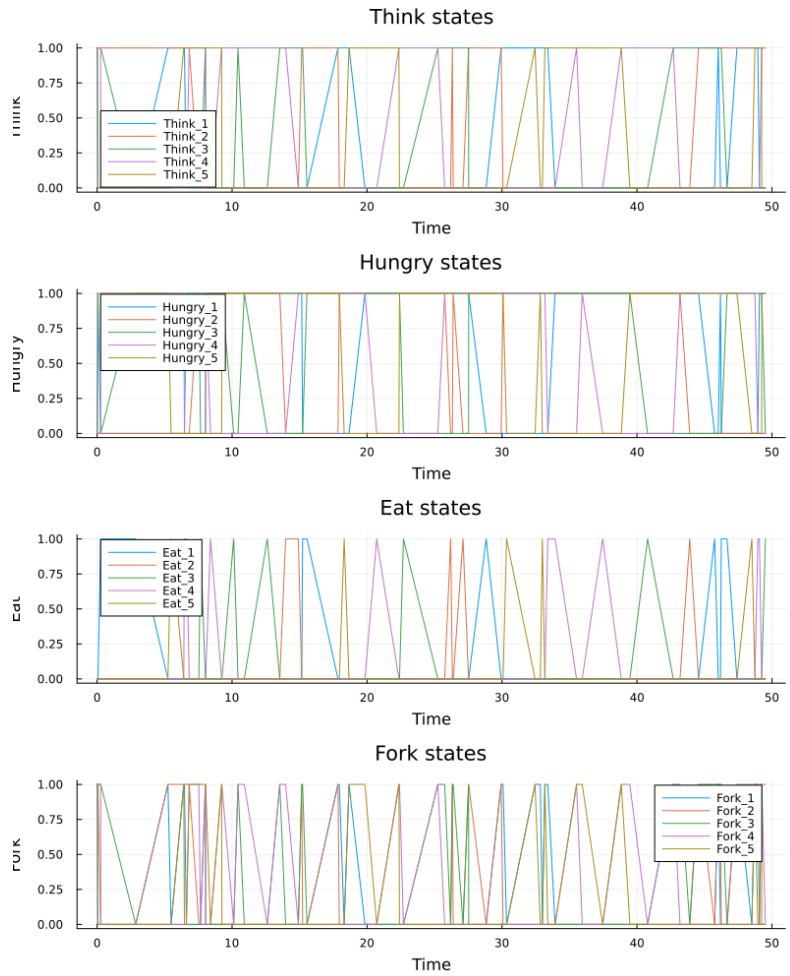

Рисунок 9: График

4.5 Выводы

- Классическая сеть демонстрирует deadlock (философы застревают в состоянии Hungry)
- Сеть с арбитром успешно предотвращает deadlock (система продолжает работать)5. Анимация работы сети Петри для задачи об обедающих философах

5.1 Введение

Данный скрипт создаёт анимацию, показывающую динамику изменения маркировки в классической сети Петри. Анимация позволяет наглядно увидеть, как фишки перемещаются между позициями и как возникает deadlock.

5.2 Параметры

- `N = 3` — количество философов (меньше для лучшей визуализации)
- `tmax = 30.0` — время моделирования

5.4 Построение классической сети

```
net, u0, names = build_classical_network(N)
```

5.5 Стохастическая симуляция

Фиксируем seed для воспроизводимости результатов.

```
Random.seed!(123)
df = simulate_stochastic(net, u0, tmax)
```

5.6 Создание анимации

Каждый кадр анимации представляет собой столбчатую диаграмму текущей маркировки. По оси X — позиции сети, по оси Y — количество фишек.

```
anim = @animate for row in eachrow(df)
    u = [row[col] for col in propertynames(row) if col != :time]
    bar(
        1:length(u), u,
        legend = false,
        ylims = (0, maximum(u0) + 1),
        xlabel = "Позиция",
        ylabel = "Фишки",
        title = "Время = $(round(row.time, digits=2))",
    )
    xticks!(1:length(u), string.(names), rotation = 45)
end
```

5.7 Интерпретация

На анимации можно наблюдать: - Перемещение фишек между позициями Think, Hungry, Eat и Fork - В определённый момент в классической модели наступает deadlock — маркировка перестаёт изменяться, все философы застревают в состоянии Hungry

Это наглядно демонстрирует проблему взаимной блокировки в параллельных системах.

6. Сравнительный анализ классической сети Петри и сети с арбитром

6.1 Введение

Данный скрипт строит сравнительный отчёт по двум моделям задачи об обедающих философах. Анализируется ключевой показатель — количество философов в состоянии “Ест” (Eat) в каждый момент времени.

6.2 Загрузка данных

Используются ранее сохранённые результаты моделирования.

```
df_classic = CSV.read(datadir("dining_classic.csv"), DataFrame)
df_arbiter = CSV.read(datadir("dining_arbiter.csv"), DataFrame)
```

6.4 Параметры

```
N = 5
```

6.5 Построение графиков для классической сети

Показываем динамику состояния Eat для каждого философа.

```
p1 = plot(
    xlabel = "Время",
    ylabel = "Ест (1/0)",
    title = "Классическая сеть (без арбитра)",
    legend = :topright,
    legendtitle = "Философ",
)

for i = 1:N
    col = Symbol("Eat_$i")
    plot!(p1, df_classic.time, df_classic[:, col], label = "Ф $i")
end
```

6.6 Построение графиков для сети с арбитром

```
p2 = plot(
    xlabel = "Время",
    ylabel = "Ест (1/0)",
    title = "Сеть с арбитром",
    legend = :topright,
    legendtitle = "Философ",
)

for i = 1:N
    col = Symbol("Eat_$i")
    plot!(p2, df_arbiter.time, df_arbiter[:, col], label = "Ф $i")
end
```

6.7 Объединение графиков

Размещаем оба графика один под другим для удобного сравнения.

```
p_final = plot(p1, p2, layout = (2, 1), size = (800, 600))
savefig(plotsdir("final_report.png"))
```

```
println("Отчёт сохранён в plots/final_report.png")
```

6.8 Интерпретация результатов

Посмотрим на график (рис. 10)

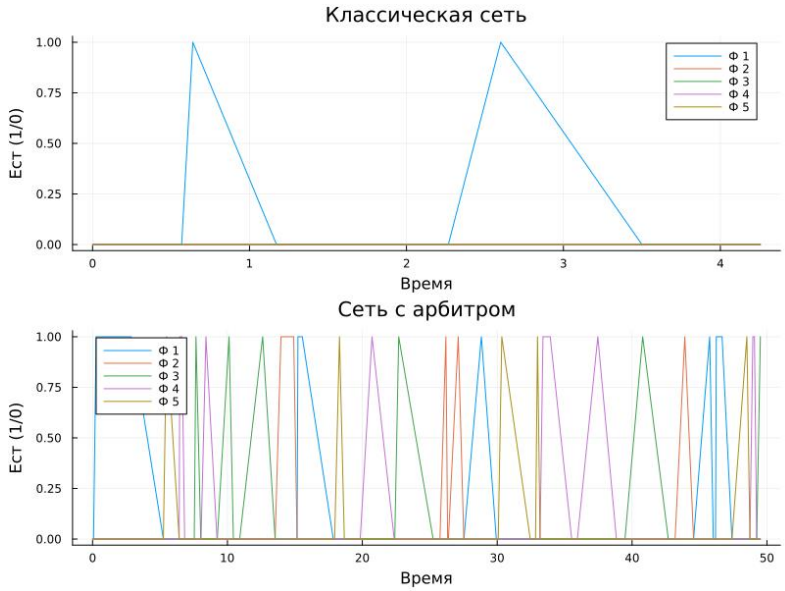

Рисунок 10: График

6.8.1 Классическая сеть (верхний график)

- Через некоторое время все линии Eat_i падают до нуля и остаются на нуле
- Это свидетельствует о наступлении deadlock — философы больше не могут есть
- Система полностью блокируется

6.8.2 Сеть с арбитром (нижний график)

- Линии Eat_i колеблются на протяжении всего времени моделирования
- Всегда есть хотя бы один философ, который ест
- Отсутствуют длительные периоды бездействия всех философов
- Система остаётся “живой” и не блокируется

6.9 Выводы

1. Классическая постановка задачи обедающих философов неизбежно приводит к deadlock
2. Введение арбитра (дополнительной позиции с N-1 фишками) эффективно предотвращает взаимную блокировку

3. Сети Петри являются удобным инструментом для моделирования и анализа проблем синхронизации в параллельных системах

7. Вывод

Сети Петри являются удобным и наглядным инструментом для моделирования параллельных систем. Задача об обедающих философах наглядно демонстрирует проблему deadlock, а модификация с арбитром показывает эффективный способ её решения. Стохастическое моделирование позволяет исследовать динамику системы в непрерывном времени.